

Detecția și Clasificarea Obiectelor în Semnale Radio Zgomotoase folosind Arhitecturi Convoluționale Reziduale și Clasificatori Clasici

Apostol Tudor

Grupa 333

Facultatea de Matematică și Informatică, Universitatea din București

E-mail: tudor-florian.apostol@s.unibuc.ro / tudor.apostol@gmail.com

Iunie 2026

1 Titlu și Introducere

În cadrul acestui proiect, abordăm o problemă propusă pe platforma Kaggle, unde sarcina constă în clasificarea imaginilor de semnale radio zgomotoase în funcție de numărul de obiecte conținute. Problema este definită ca o clasificare în 5 clase distincte (corespunzătoare unui număr de la 1 la 5 obiecte). Setul de date furnizat este compus dintr-un set de antrenare de 15.500 de imagini și un set de test de 5.500 de imagini, fiecare având rezoluția de 55 pe 128 pixeli și 4 canale de culoare (RGBA).

În această documentație se va prezenta metodologia completă de rezolvare a problemei, începând de la fazele inițiale de preprocesare și augmentare a imaginilor, continuând cu analiza detaliată a celor două metode propuse (clasificatorul clasic SVM și rețeaua neuronală CustomCNN) și încheind cu analiza rezultatelor experimentale și a strategiei finale bazate pe un ansamblu de modele.

2 Preprocesare și Augmentarea Datelor

Faza de pregătire a datelor reprezintă o etapă critică în orice proiect de învățare automată, având rolul de a ajuta modelul să învețe mai ușor și de a preveni memorarea zgomotului de fundal. În acest scop, a fost dezvoltată o clasă pentru gestionarea setului de date, denumită `SignalDataset` în cadrul scriptului `data_loader.py`.

Prima decizie a fost conversia spațiului de culori. Deoarece canalul Alpha (transparența) era constant pe întregul set de imagini, acesta nu aducea nicio informație utilă, dar mărea inutil consumul de memorie. Prin urmare, imaginile originale de 4 canale (RGBA) au fost convertite la formatul RGB cu 3 canale prin intermediul bibliotecii PIL:

```
image = Image.open(img_path).convert('RGB')
```

Această conversie a redus memoria utilizată cu 25%, un aspect critic având în vedere rularea locală pe un MacBook Air cu 8GB RAM.

În faza următoare, pixelii imaginilor au fost transformați în matrici de numere (numite tensori în biblioteca PyTorch) și scalați (normalizați) pentru a aduce toate valorile de culoare din intervalul inițial (0 pentru negru și 255 pentru alb) într-un interval numeric mai mic, cuprins între -1 și 1. Această transformare păstrează toate culorile și nuanțele originale din poze, dar ajută calculatorul să facă calculele mult mai rapid și mai stabil.

Pentru a spori capacitatea de generalizare a rețelei convoluționale, s-a proiectat un set de augmentări geometrice aplicat aleatoriu pe setul de antrenare:

- `RandomHorizontalFlip(p=0.5)` – oglindirea orizontală a imaginii.

- `RandomVerticalFlip(p=0.5)` – oglindirea verticală a imaginii.
- `RandomRotation(degrees=10)` – rotirea ușoară a imaginii cu cel mult 10 grade pentru a simula mici variații de orientare a semnalelor în spațiu.

Este important de menționat decizia de a evita tehnici comune de augmentare a culorii, cum ar fi modificarea strălucirii sau a contrastului (`ColorJitter`). În semnalele radio reprezentate vizual, intensitatea culorilor și nivelul contrastului reprezintă în mod direct puterea fizică a semnalului. Alterarea acestor proprietăți ar fi distrus informația fizică reală din spatele datelor. Pentru seturile de validare și testare, imaginile au fost păstrate nemodificate, aplicându-se doar normalizarea standard.

Metoda 1: Model Clasic (SVM) bazat pe Feature Engineering Algoritmii clasici de învățare automată nu pot procesa eficient imagini brute direct la dimensiuni mari. O imagine de 55 pe 128 pixeli pe 3 canale RGB reprezintă o intrare formată din peste 21.000 de valori individuale. În lipsa unui extractor de trăsături, modelele clasice eșuează rapid. Din acest motiv, s-a implementat o etapă de extragere manuală a caracteristicilor relevante.

Scripturile noastre extrag un vector reprezentativ format din **109 caracteristici globale** pentru fiecare imagine:

1. **Histograme de culoare (96 caracteristici):** Se măsoară distribuția culorilor în imagine pe 32 de intervale pentru fiecare canal (R, G, B), normalizată pentru a asigura stabilitatea.
2. **Statistici pe canale (9 caracteristici):** Media pixelilor, deviația standard (împrăștierea) și asimetria calculate independent pentru canalele R, G și B.
3. **Statistici pe imaginea de gri (4 caracteristici):** Media, deviația standard, valoarea minimă și maximă obținute după conversia imaginii la scară de gri (grayscale).

Matricea de caracteristici rezultată este standardizată folosind instrumentul `StandardScaler`. Această etapă scalează toate caracteristicile la același nivel de mărime (scăzând valoarea medie și împărțind la abaterea standard), asigurând că nicio caracteristică nu domină calculele doar pentru că are valori numere mai mari din fabrică.

Pentru clasificare, s-a utilizat algoritmul SVM cu nucleu RBF. Optimizarea s-a realizat prin căutare pe grilă (Grid Search) pe parametri de regularizare, evaluarea fiind făcută prin validare încrucișată în 5 pliuri (folduri).

Cel mai bun model SVM a obținut o acuratețe medie de doar **27.50%**. Eșecul acestei metode arată că histogramele și statisticile globale pierd complet informația spațială (unde se află obiectele). Deoarece zgomotul este uniform distribuit în imagini, statisticile globale ale tuturor claselor sunt extrem de similare, făcând algoritmul clasic incapabil să le distingă.

3 Metoda 2: Deep Learning (CustomCNN)

Pentru a depăși limitările trăsăturilor globale extrase manual, am folosit o Rețea Neuronală Convoluțională. Acest model funcționează ca un sistem vizual artificial: scanează imaginea în detaliu și învață singur să identifice forme locale (linii, colțuri și contururi), ignorând zgomotul aleatoriu de fundal.

Pentru a asigura o transmitere eficientă a informației prin straturile rețelei, am definit piesa de bază numită `ResidualBlock` (bloc rezidual). Fiecare bloc conține straturi de convoluție de 3 pe 3, Batch Normalization (normalizarea datelor) și funcția de activare ReLU. O conexiune de tip scurtătură (shortcut) adună direct intrarea originală la ieșire, prevenind pierderea detaliilor pe măsură ce rețeaua devine mai adâncă.

Arhitectura rețelei `CustomCNN` este structurată astfel:

- **Blocul 1:** Bloc Rezidual (3 la 64 de canale) → Max-Pooling (reducerea dimensiunii la jumătate) → Dropout (0.25 probabilitate de dezactivare a neuronilor pentru prevenirea memorării).

- **Blocul 2:** Bloc Rezidual (64 la 128 de canale) → Max-Pooling → Dropout (0.25).
- **Blocul 3:** Bloc Rezidual (128 la 256 de canale) → Max-Pooling → Dropout (0.25).
- **Blocul 4:** Bloc Rezidual (256 la 512 de canale) → Pooling Adaptiv Mediu – reduce dimensiunea spațială direct la 1 pe 1 pixel, extrăgând un vector final stabil de 512 elemente.
- **Clasificatorul:** Strat-uri dense de dimensiuni finale reduse treptat la cele 5 clase, regularizate prin Dropout puternic (0.5 și 0.3).

Rularea s-a efectuat pe un sistem MacBook Air M2. Pentru a accelera procesul, calculele au fost mutate pe placa grafică integrată a procesorului. Din cauza resurselor limitate de memorie (8GB RAM), codul a fost optimizat prin dezactivarea proceselor secundare de încărcare și prin curățarea periodică a memoriei video după procesarea fiecărui pliuri (fold), prevenind astfel blocarea sistemului.

4 Setări Experimentale, Hyperparameter Tuning și Rezultate

Evaluarea corectă a modelelor a fost realizată prin validare încrucișată stratificată în 5 pliuri (folduri) (Stratified 5-Fold Cross-Validation). Procesul de optimizare a rețelei CustomCNN a implicat trei ruli experimentale distincte (run-uri). Optimizarea s-a realizat folosind algoritmul AdamW cu o rată de învățare inițială de 0.001 și o descreștere progresivă a acesteia prin funcția `CosineAnnealingLR`. Funcția de cost utilizată a fost `CrossEntropyLoss` cu netezirea etichetelor (`label_smoothing`).

Tabela 1: Rezultatele Grid Search pentru modelul SVM

Parametrul C	Coeficientul γ	Acuratețe Medie CV	Deviație Standard
0.1	'scale'	26.21%	$\pm 0.80\%$
0.1	'auto'	26.21%	$\pm 0.86\%$
1.0 (Optimizat)	'scale'	27.50%	$\pm 0.26\%$
1.0	'auto'	27.47%	$\pm 0.47\%$
10.0	'scale'	26.96%	$\pm 0.67\%$
10.0	'auto'	27.10%	$\pm 0.72\%$
100.0	'scale'	25.28%	$\pm 0.58\%$
100.0	'auto'	25.54%	$\pm 0.46\%$

Tabela 2: Istoricul antrenărilor CustomCNN

Parametru	Run 1	Run 2	Run 3 (Final)
Epoci Max	50	50	60
Label Smoothing	0.10	0.10	0.05
Early Stopping Patience	8 epoci	15 epoci	15 epoci
Acuratețe Medie CV	68.79%	68.90%	73.79%
Deviație Std CV	$\pm 3.91\%$	$\pm 0.21\%$	$\pm 0.36\%$

În Run 1, o oprire rapidă din instruire (Early Stopping la 8 epoci fără îmbunătățire) a oprit prematur modelul pe anumite pliuri (folduri). Creșterea acestei limite la 15 epoci în Run 2 a stabilizat procesul, reducând variațiile între pliuri (folduri). În final, Run 3 a demonstrat că o netezire mai mică a etichetelor (Label Smoothing = 0.05) combinată cu 60 de epoci a permis modelului să extragă trăsături mai fine, ridicând acuratețea medie la **73.79%** (cu rezultate per pliuri/fold între 73.16% și 74.19%).

5 Analiza Erorilor (Matricele de Confuzie)

Analiza performanțelor s-a realizat prin interpretarea matricelor de confuzie generate pe setul de validare (un tabel standard care arată grafic ce clase tinde să confunde modelul). Pentru modelul baseline SVM,

se observă o orientare eronată masivă către clasa 1, confirmând ineficiența trăsăturilor globale în prezența zgomotului.

În cazul modelului CustomCNN (Run 3), matricea de confuzie prezintă o diagonală principală puternic conturată, indicând că majoritatea deciziilor sunt corecte. Totuși, se observă două categorii de erori sistematice:

- **Confuzia între clase adiacente:** Modelul tinde uneori să confunde clasa reală k cu clasa $k - 1$ sau $k + 1$. Zgomotul de fundal poate acoperi marginile unui obiect, făcându-l invizibil (sub-numărare) sau, dimpotrivă, poate crea zgomot izolat ce pare a fi un semnal valid (supra-numărare).
- **Performanța redusă la clasa 5:** Clasificată cel mai des ca fiind clasa 4. Întrucât rezoluția imaginii este foarte mică, prezența a 5 obiecte în interiorul acestui spațiu îngust determină suprapunerea și lipirea lor vizuală (fenomenul de *overlap*), modelul interpretându-le ca fiind doar 4.

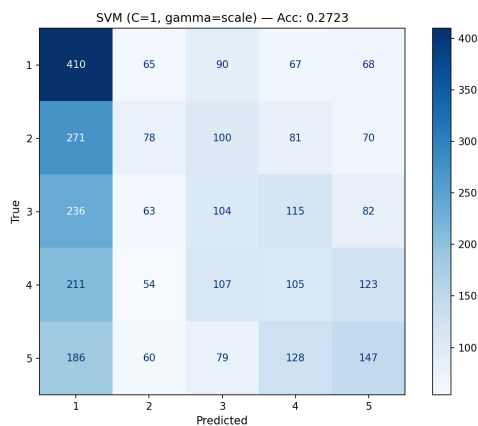


Figura 1: Matricea de confuzie SVM baseline.

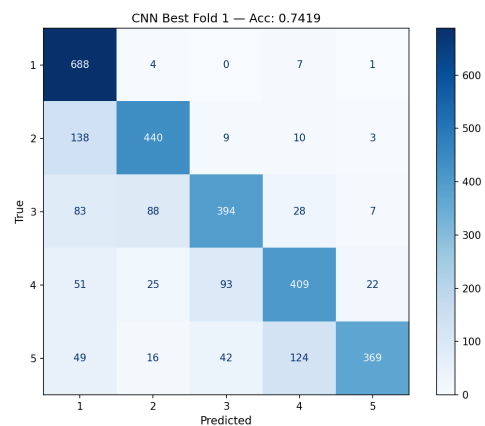


Figura 2: Matricea CustomCNN (Fold 1, Run 3).

6 Concluzii și Strategia de Submisie (Ansamblu de Modele)

Studiul comparativ realizat în acest proiect demonstrează în mod clar superioritatea algoritmilor de Deep Learning pe date complexe de semnale radio în raport cu metodele tradiționale. Trecerea de la SVM (27.50%) la CustomCNN (73.79%) a însemnat o creștere de peste **46%** în acuratețea de Cross-Validation.

Pentru a obține cel mai bun rezultat pe setul de test de pe platforma Kaggle, s-a folosit o metodă bazată pe un ansamblu de modele (Ensemble). În loc să utilizăm o singură rețea, s-au combinat deciziile de la toate cele 5 pliuri (folduri) antrenate în Run 3.

Pentru fiecare imagine de test nouă, se extrag probabilitățile estimate de cele 5 modele pentru fiecare clasă și se calculează media lor. Clasa finală prezisă este cea care obține cea mai mare medie a probabilităților dintre cele 5 opțiuni posibile. Această abordare a eliminat variațiile individuale ale fiecărui model și a asigurat o performanță stabilă pe setul de test. Modelul a obținut un scor final oficial pe datele private de **75.78%** (Private Score = 0.75781), respectiv de **76.36%** pe clasamentul public preliminar (Public Score = 0.76363), oferind autorului locul 79 în clasamentul final.